



Security Audit

Report for lista - rwa - contracts

Date: October 17, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Security Issue	4
2.1.1 USDC depeg risk	4
2.1.2 Incorrect rounding direction	6
2.1.3 Potential DoS due to lack of timely update of <code>userTotalAssets</code> in function <code>requestWithdraw()</code>	8
2.1.4 Potential inflation attack due to precision loss and lack of minimum checks	9
2.1.5 Reward distribution distortion due to period extension by unvested rewards	10
2.2 Recommendation	11
2.2.1 Add zero address check for <code>_bot</code> in function <code>initialize()</code>	11
2.2.2 Revise the check of <code>_feeRate</code>	11
2.3 Note	12
2.3.1 <code>AsyncVault</code> is trusted and compliable with protocol design	12
2.3.2 Potential centralization risks	12

Report Manifest

Item	Description
Client	lista - dao
Target	lista - rwa - contracts

Version History

Version	Date	Description
1.0	October 17, 2025	First release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository ¹ of lista-rwa-contracts of lista-dao.

This RWA investment module enables USD1 holders to gain exposure to off-chain asset yields through an integrated contract system. The RWAEarnPool manages deposits, withdrawal requests, and the distribution of earnings. The RWAAdapter interfaces with the Centrifuge vault by converting USD1 to USDC, depositing the funds, and, upon redemption, withdrawing USDC, converting it back to USD1, and returning the proceeds to RWAEarnPool. The OTCManager facilitates two-way conversions between USD1 and USDC, supporting both deposit and redemption flows. Collectively, these contracts provide a structured framework for accessing RWA yields while maintaining centralized accounting, conversion, and settlement logic.

Note this audit only focuses on the smart contracts in the following directories/files:

- src/rwa/*

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
lista-rwa-contracts	Version 1	78a3aff32bd13da3089dcadb9bd94f9458285
	Version 2	b86a34731ea06b88bb426ad172b85e384a282bc6

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

¹<https://github.com/lista-dao/lista-rwa-contracts>

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency
- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ² and Common Weakness Enumeration ³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **five** potential security issues. Besides, we have **two** recommendations and **two** notes.

- Medium Risk: 1
- Low Risk: 4
- Recommendation: 2
- Note: 2

ID	Severity	Description	Category	Status
1	Medium	USDC depeg risk	Security Issue	Confirmed
2	Low	Incorrect rounding direction	Security Issue	Confirmed
3	Low	Potential DoS due to lack of timely update of <code>userTotalAssets</code> in function <code>requestWithdraw()</code>	Security Issue	Fixed
4	Low	Potential inflation attack due to precision loss and lack of minimum checks	Security Issue	Fixed
5	Low	Reward distribution distortion due to period extension by unvested rewards	Security Issue	Confirmed
6	-	Add zero address check for <code>_bot</code> in function <code>initialize()</code>	Recommendation	Fixed
7	-	Revise the check of <code>_feeRate</code>	Recommendation	Fixed
8	-	<code>AsyncVault</code> is trusted and compliable with protocol design	Note	-
9	-	Potential centralization risks	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 USDC depeg risk

Severity Medium

Status Confirmed

Introduced by Version 1

Description The protocol converts all deposited `USD1` into `USDC` for yield farming, then converts the `USDC` back to `USD1` for withdrawals and reward distribution. This design assumes `USD1` and `USDC` maintain price parity, but fails to account for potential `USDC` depegging scenarios.

If `USDC` depegs and trades below `USD1`'s value, the vault's `USDC` holdings become insufficient to purchase the required amount of `USD1` needed to cover user principal and earned reward during redemptions. This creates a liquidity shortfall that could result in denial of service for withdrawal requests.

Additionally, yield earned in `USDC` is only reflected as increased `USD1` share value in the accounting system until the privileged bot executes the actual conversion, exposing users to price volatility risk during this interim period.

```
290 /**
291  * @dev get conversion from USD1 to USDC
292  * @param amount The amount of USD1 to convert
293  * @return converted amount in USDC
294  */
295 function USD1ToUSDC(uint256 amount) public view returns (uint256) {
296     return amount - amount.mulDiv(toUSDCLossRate, PRECISION);
297 }
298
299 /**
300  * @dev get conversion from USDC to USD1
301  * @param amount The amount of USDC to convert
302  * @return converted amount in USD1
303  */
304 function USDCtoUSD1(uint256 amount) public view returns (uint256) {
305     return amount - amount.mulDiv(toUSD1LossRate, PRECISION);
306 }
```

Listing 2.1: src/rwa/RWAAdapter.sol

```
70 /* EXTERNAL FUNCTIONS */
71 /**
72  * @dev swaps tokens from adapter to OTC wallet.
73  * @param token The address of the token to swap (USD1 or USDC).
74  * @param amount The amount of tokens to swap.
75  */
76 function swapToken(address token, uint256 amount) external {
77     require(msg.sender == adapter, "Only adapter can call this function");
78     require(token == USD1 || token == USDC, "Invalid token");
79     IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
80     IERC20(token).safeTransfer(otcWallet, amount);
81 }
82
83 /**
84  * @dev transfers tokens from OTC wallet to adapter.
85  * @param token The address of the token to transfer (USD1 or USDC).
86  * @param amount The amount of tokens to transfer.
87  */
88 function transferToAdapter(address token, uint256 amount) external onlyRole(BOT) {
89     require(amount > 0, "Amount must be greater than zero");
90     require(token == USD1 || token == USDC, "Invalid token");
91     IERC20(token).safeTransfer(adapter, amount);
92 }
```

Listing 2.2: src/rwa/OTCManager.sol

Impact This can result in insufficient liquidity for user withdrawals, potentially resulting in permanent capital loss and denial of service.

Suggestion Revise the logic accordingly.

Feedback from the project All exchanges are conducted via OTC, eliminating any depeg risk.

2.1.2 Incorrect rounding direction

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description The protocol allows users to specify either an asset amount or a share amount when depositing or withdrawing. When the user specifies shares for deposit, the required asset amount to deposit is calculated using floor rounding. Similarly, when the user specifies assets for withdrawal, the required shares to be burned are also determined with floor rounding. This approach systematically favors the user, as they can pay slightly less or burn fewer shares than intended, creating a rounding bias against the protocol.

```
155 function deposit(uint256 amount, uint256 shares, address receiver) external whenNotPaused
    nonReentrant {
156     require(amount > 0 || shares > 0, "amount and shares is zero");
157     require(receiver != address(0), "receiver is zero address");
158     require(isInWhitelist(receiver), "receiver not in whitelist");
159
160     // calculate shares or amount
161     if (amount > 0) {
162         shares = convertToShares(amount);
163     } else {
164         amount = convertToAssets(shares);
165     }
166     // mint shares to user
167     _mint(receiver, shares);
168     userTotalAssets += amount;
169     // transfer asset from user
170     IERC20(asset).safeTransferFrom(msg.sender, adapter, amount);
171
172     emit Deposit(receiver, amount);
173 }
```

Listing 2.3: src/rwa/RWAEarnPool.sol

```
334 function convertToAssets(uint256 shares) public view returns (uint256) {
335     // assets = shares * totalAssets / totalSupply
336     return shares.mulDiv(totalAssets() + 1, totalSupply + 1);
337 }
```

Listing 2.4: src/rwa/RWAEarnPool.sol

```
181 function requestWithdraw(uint256 amount, uint256 shares, address receiver) external
    whenNotPaused nonReentrant {
182     require(amount > 0 || shares > 0, "amount and shares is zero");
183     require(receiver != address(0), "receiver is zero address");
184
185     // calculate shares or amount
186     if (amount > 0) {
187         shares = convertToShares(amount);
188     } else {
```

```
189     amount = convertToAssets(shares);
190 }
191
192 require(balanceOf[msg.sender] >= shares, "insufficient shares");
193
194 uint256 feeShares;
195 // charge withdraw fee
196 if (withdrawFeeRate > 0 && feeReceiver != address(0)) {
197     // feeShares = shares * withdrawFeeRate / PRECISION
198     feeShares = shares.mulDiv(withdrawFeeRate, PRECISION);
199     if (feeShares > 0) {
200         // transfer fee shares to feeReceiver
201         _transfer(msg.sender, feeReceiver, feeShares);
202
203         shares -= feeShares;
204         amount = convertToAssets(shares);
205     }
206 }
207
208 // burn shares from user
209 _burn(msg.sender, shares);
210
211 // if new day, increase batch id
212 // if currentBatchId == confirmedBatchId, increase batch id too
213 uint256 day = block.timestamp / 1 days;
214 if (day > lastDay) {
215     lastDay = day;
216     ++currentBatchId;
217 } else if (currentBatchId == confirmedBatchId) {
218     ++currentBatchId;
219 }
220
221 // add withdraw request to user
222 userWithdrawalRequests[receiver].push(
223     WithdrawalRequest({ batchId: currentBatchId, amount: amount, withdrawTime: block.timestamp
224         })
225 );
226
227 // update total withdraw amount in batch
228 totalWithdrawAmountInBatch[currentBatchId] += amount;
229 // update user total assets
230 userTotalAssets -= amount;
231
232 emit RequestWithdraw(msg.sender, receiver, currentBatchId, shares, amount, feeShares);
233 }
```

Listing 2.5: src/rwa/RWAEarnPool.sol

```
324 function convertToShares(uint256 assets) public view returns (uint256) {
325     // shares = assets * totalSupply / totalAssets
326     return assets.mulDiv(totalSupply + 1, totalAssets() + 1);
327 }
```

Listing 2.6: src/rwa/RWAEarnPool.sol

Impact Over time, this rounding bias may lead to small but cumulative value leakage from the protocol to users.

Suggestion Revise the logic to ensure the rounding direction favors the protocol.

Feedback from the project The issue has been acknowledged.

2.1.3 Potential DoS due to lack of timely update of `userTotalAssets` in function `requestWithdraw()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the function `requestWithdraw()`, the protocol subtracts the withdrawal amount from `userTotalAssets` without first refreshing it using the latest value from function `totalAssets()`. As rewards accrue over time, `userTotalAssets` may become outdated, diverging from the actual assets held. When users request withdrawals under these conditions, the protocol may attempt to subtract more than the recorded balance, creating an underflow and preventing subsequent withdrawals.

```
181 function requestWithdraw(uint256 amount, uint256 shares, address receiver) external
    whenNotPaused nonReentrant {
182     require(amount > 0 || shares > 0, "amount and shares is zero");
183     require(receiver != address(0), "receiver is zero address");
184
185     // calculate shares or amount
186     if (amount > 0) {
187         shares = convertToShares(amount);
188     } else {
189         amount = convertToAssets(shares);
190     }
191
192     require(balanceOf[msg.sender] >= shares, "insufficient shares");
193
194     uint256 feeShares;
195     // charge withdraw fee
196     if (withdrawFeeRate > 0 && feeReceiver != address(0)) {
197         // feeShares = shares * withdrawFeeRate / PRECISION
198         feeShares = shares.mulDiv(withdrawFeeRate, PRECISION);
199         if (feeShares > 0) {
200             // transfer fee shares to feeReceiver
201             _transfer(msg.sender, feeReceiver, feeShares);
202
203             shares -= feeShares;
204             amount = convertToAssets(shares);
205         }
206     }
207
208     // burn shares from user
209     _burn(msg.sender, shares);
210 }
```

```
211 // if new day, increase batch id
212 // if currentBatchId == confirmedBatchId, increase batch id too
213 uint256 day = block.timestamp / 1 days;
214 if (day > lastDay) {
215     lastDay = day;
216     ++currentBatchId;
217 } else if (currentBatchId == confirmedBatchId) {
218     ++currentBatchId;
219 }
220
221 // add withdraw request to user
222 userWithdrawalRequests[receiver].push(
223     WithdrawalRequest({ batchId: currentBatchId, amount: amount, withdrawTime: block.timestamp
224         })
225 );
226
227 // update total withdraw amount in batch
228 totalWithdrawAmountInBatch[currentBatchId] += amount;
229 // update user total assets
230 userTotalAssets -= amount;
231
232 emit RequestWithdraw(msg.sender, receiver, currentBatchId, shares, amount, feeShares);
233 }
```

Listing 2.7: src/rwa/RWAEarnPool.sol

```
291 function notifyInterest(uint256 amount) external {
292     require(msg.sender == adapter, "only adapter can call");
293
294     // update user total assets
295     userTotalAssets = totalAssets();
296
297     // update period rewards and period start
298     periodRewards = getUnvestedAmount() + amount;
299     periodStart = block.timestamp;
300
301     emit NotifyInterest(amount);
302 }
```

Listing 2.8: src/rwa/RWAEarnPool.sol

Impact Users may be unable to withdraw their deposited USD1.

Suggestion Update `userTotalAssets` in the function `requestWithdraw()` in a timely manner.

2.1.4 Potential inflation attack due to precision loss and lack of minimum checks

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `RWAEarnPool`, the conversion between assets and shares is based on division between `totalSupply` and `totalAssets`. Since `totalAssets` may increase over time

through periodic interest updates, this calculation can suffer from precision loss when users deposit very small amounts. In such cases, the protocol may issue zero shares while still accepting the deposit, enabling attackers to repeatedly inflate the value of existing shares. Without explicit minimum-share checks in functions `deposit()` and `requestWithdraw()`, this imbalance can be exploited to drain the pool.

```
324 function convertToShares(uint256 assets) public view returns (uint256) {
325     // shares = assets * totalSupply / totalAssets
326     return assets.mulDiv(totalSupply + 1, totalAssets() + 1);
327 }
```

Listing 2.9: src/rwa/RWAEarnPool.sol

```
161 if (amount > 0) {
162     shares = convertToShares(amount);
163 } else {
164     amount = convertToAssets(shares);
165 }
```

Listing 2.10: src/rwa/RWAEarnPool.sol

Impact This vulnerability enables potential inflation attacks that could drain underlying tokens from the pool.

Suggestion Add checks to ensure that the deposited and withdrawal shares are greater than the minimum share amount.

2.1.5 Reward distribution distortion due to period extension by unvested rewards

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description In the contract `RWAEarnPool`, the function `notifyInterest()` adds unvested rewards to the next distribution cycle and resets the period start, even a small additional interest amount can extend the ongoing vesting period. This behavior shifts the reward schedule, reducing the effective yield for users who expected to exit after a fixed duration, as their rewards are spread over a longer timeframe than originally intended.

```
291 function notifyInterest(uint256 amount) external {
292     require(msg.sender == adapter, "only adapter can call");
293
294     // update user total assets
295     userTotalAssets = totalAssets();
296
297     // update period rewards and period start
298     periodRewards = getUnvestedAmount() + amount;
299     periodStart = block.timestamp;
300
301     emit NotifyInterest(amount);
302 }
```

Listing 2.11: src/rwa/RWAEarnPool.sol

Impact This design results in delayed reward distribution and potential loss of users' rewards.

Suggestion Revise the reward accrual logic to prevent unvested rewards from extending the ongoing distribution period.

Feedback from the project The frequency of `notifyInterest` will be managed by the `Manager` and the `Bot`.

2.2 Recommendation

2.2.1 Add zero address check for `_bot` in function `initialize()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `OTCManager` contract, the `initialize()` function assigns roles to several addresses, including `_bot`. Currently, it validates `_admin`, `_manager`, `_adapter`, and `_otcWallet` against the zero address but does not check `_bot`. Assigning the `BOT` role to the zero address is meaningless and could lead to misconfiguration or unexpected behavior.

```

48 function initialize(
49     address _admin,
50     address _manager,
51     address _bot,
52     address _adapter,
53     address _otcWallet
54 ) external initializer {
55     require(_admin != address(0), "Admin is zero address");
56     require(_manager != address(0), "Manager is zero address");
57     require(_adapter != address(0), "Adapter is zero address");
58     require(_otcWallet != address(0), "OTC wallet is zero address");
59
60     __AccessControlEnumerable_init();
61
62     _grantRole(DEFAULT_ADMIN_ROLE, _admin);
63     _grantRole(MANAGER, _manager);
64     _grantRole(BOT, _bot);
65
66     adapter = _adapter;
67     otcWallet = _otcWallet;
68 }

```

Listing 2.12: src/rwa/OTCManager.sol

Suggestion Add the check to ensure the parameter `_bot` is not the zero address.

2.2.2 Revise the check of `_feeRate`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the [RWAAAdapter](#) contract, the `setFeeRate()` function is intended to restrict `_feeRate` to a maximum of 10% according to the comment. However, the current check only ensures `_feeRate <= PRECISION`, which does not enforce the intended 10% cap. This inconsistency between the code and the comment may lead to setting a fee rate higher than intended.

```
258 function setFeeRate(uint256 _feeRate) external onlyRole(MANAGER) {  
259     require(_feeRate <= PRECISION, "feeRate too high"); // max 10%  
260     feeRate = _feeRate;  
261 }
```

Listing 2.13: `src/rwa/RWAAAdapter.sol`

Suggestion Revise the check of `_feeRate`.

2.3 Note

2.3.1 AsyncVault is trusted and compliant with protocol design

Introduced by [Version 1](#)

Description In the current design, user deposits in [USD1](#) are swapped via [OTCManager](#) into [USDC](#) and then deposited into [AsyncVault](#), which is documented as an [ERC7540](#) based vault. The vault itself is out of scope for this audit and is assumed to be trustworthy. However, it must remain compliant with the protocol's design assumptions and security model. Since [ERC7540](#) extends [ERC4626](#), the vault implementation should ensure that common [ERC4626](#) attack surfaces are either mitigated or explicitly ruled out through proper configuration.

2.3.2 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., [manager](#)) can conduct sensitive operations, which introduces potential centralization risks. For instance, the [manager](#) is granted the authority to withdraw deposited assets. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

